



Categorical Encoding in R: Concept Hierarchies vs One-Hot Encoding

Categorical Mapping

Among the techniques for encoding categorical variables, **categorical mapping** stands out for allowing the construction of concept hierarchies from existing attributes. Rather than simply converting categories into numbers, this approach systematically groups related values into higher-level categories, preserving data semantics and enhancing the analytical power of the model.

To consolidate this concept, the next slide presents a **practical example implemented in R**, where we will apply categorical mapping step by step on a real dataset — making it clear how each stage of the process contributes to a more structured and interpretable representation of variables.

Category Identification

Analyze the unique values of the categorical attribute

Grouping Definition

Establish higher-level categories based on similarity or semantics

Mapping Creation

Associate each original value with its corresponding category

Hierarchy Validation

Verify that the mapping makes sense within the domain context

Example of Categorical Mapping

With the four steps of the process well defined, it is time to put them into practice. The example below concretely shows how specific values of a categorical variable are organized into a hierarchical structure — from identifying the original categories to consolidating them into higher-level groupings:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Speciessetosa	Speciesversicolor	Speciesvirginica
1	5.1	3.5	1.4	0.2	1	0	0
2	4.9	3.0	1.4	0.2	1	0	0
3	4.7	3.2	1.3	0.2	1	0	0
4	4.6	3.1	1.5	0.2	1	0	0
5	5.0	3.6	1.4	0.2	1	0	0

Notice how the more granular values are systematically grouped into intermediate categories, which in turn consolidate into even more general categories. This multilevel structuring, which we have just visualized conceptually, will be translated line by line into R code in the next slide — using the `categ_mapping` function to automate this entire process.

- ❏ The effectiveness of the mapping depends on the domain knowledge applied in the grouping definition step. In the next slide, we will see how the R implementation makes this process reproducible and scalable for large volumes of data.

Overview – Encoding

After mapping and consolidating the categories as seen in the previous example, the next step is to transform these categorical variables into a numerical format that machine learning algorithms can process. This section summarizes the core concept of encoding before we move on to the practical implementation in R code.

`categ_mapping`

Converts categorical variables into binary columns (one-hot encoding). Each level of the original variable becomes a 0/1 column. Required for algorithms that expect numerical inputs.

- ❏ **Fundamental principle for the code that follows:** Encoding must be **fitted exclusively on the training set** and only applied on the test set. New categories that appear in the test set but were not present in the training set must be handled explicitly — otherwise, the model will generate errors or produce inconsistent predictions. Keep this in mind when analyzing the implementation example in the next slide.

Categorical Encoding

With the encoding concept established, we now move to practical implementation. The example below demonstrates how `categ_mapping` from the `daltoolbox` package applies one-hot encoding to the `iris` dataset — transforming the categorical column `Species` into binary numeric columns, exactly as described in the overview.

```
library(daltoolbox)

iris <- datasets::iris

# One-hot encoding da coluna Species
cm <- categ_mapping("Species")
iris_cm <- transform(cm, iris)
print(head(iris_cm))
# Speciessetosa Speciesversicolor Speciesvirginica
#      1      0      0
#      1      0      0
#      1      0      0

# Aplicando a uma única coluna (data frame)
diris <- iris[, "Species", drop=FALSE]
iris_cm2 <- transform(cm, diris)
print(head(iris_cm2))
# Speciessetosa Speciesversicolor Speciesvirginica
#      1      0      0
```

Step by step

- Specify the categorical column in `categ_mapping("Species")`
- The result is `n` binary columns (one-hot), one per category
- Also works on a single-column data frame (`drop=FALSE`)
- Alternative: use `n-1` columns to avoid multicollinearity in linear models

📌 **Key concept:** As seen in the overview, one-hot encoding (Bishop, 2006) converts each level of the categorical variable into an independent 0/1 column — allowing numeric algorithms to process nominal variables without imposing any artificial order among the categories. Remember: the mapping must be **fitted on the training set only** and then applied to the test set.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

